

Playwright Agent Framework - Complete Usage Guide

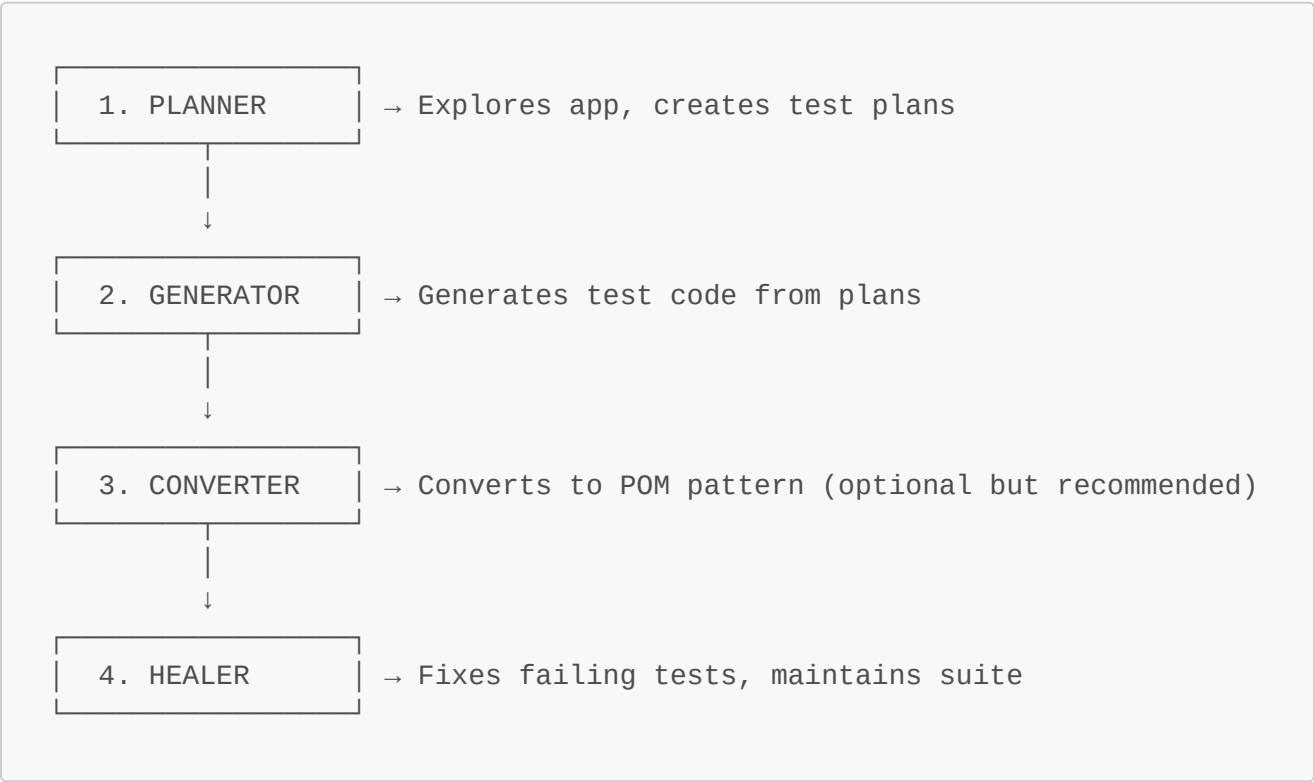
Welcome to the Playwright Agent Framework! This guide explains how to use the four specialized AI agents to build, maintain, and scale your Playwright test automation suite with industry best practices.

Table of Contents

- 1. [Overview](#)
 - 2. [The Four Agents](#)
 - 3. [Quick Start Scenarios](#)
 - 4. [Scenario 1: Starting from Scratch](#)
 - 5. [Scenario 2: Converting Existing Tests](#)
 - 6. [Scenario 3: Maintaining Tests](#)
 - 7. [Scenario 4: Expanding Test Coverage](#)
 - 8. [Scenario 5: Complete CI/CD Workflow](#)
 - 9. [Best Practices](#)
 - 10. [Troubleshooting](#)
-

Overview

The Playwright Agent Framework consists of four specialized AI agents that work together to provide a complete test automation lifecycle:



The Four Agents

🕒 Playwright Test Planner (@playwright-test-planner)

Purpose: Explore web applications and create comprehensive test plans

When to use:

- Starting a new testing project
- Need to understand application flows
- Planning test coverage strategy
- Documenting user journeys

Output: Markdown test plan files with test suites, scenarios, and steps

⚡ Playwright Test Generator (@playwright-test-generator)

Purpose: Generate executable Playwright test code from test plans

When to use:

- After creating test plans
- Need to quickly implement tests
- Want consistent test code structure
- Automating repetitive test creation

Output: TypeScript test files (.spec.ts) ready to run

🔧 Playwright POM Converter (@playwright-pom-converter)

Purpose: Refactor tests to use Page Object Model pattern with security best practices

When to use:

- Tests are becoming hard to maintain
- Need to reduce code duplication
- Want to implement security best practices
- Setting up scalable architecture
- Preparing for long-term maintenance

Output:

- Page object classes (src/pages/)
 - Test fixtures (src/fixtures/)
 - Environment variable setup
 - Comprehensive documentation
-

🩹 Playwright Test Healer (@playwright-test-healer)

Purpose: Diagnose and fix failing tests automatically

When to use:

- Tests are failing after UI changes
- Locators are broken
- Need to update selectors
- Debugging test failures
- CI/CD pipeline is failing

Output: Fixed test files with updated locators and logic

Quick Start Scenarios

Choose the scenario that matches your situation:

Scenario	Starting Point	Recommended Flow
New Project	No tests	Planner → Generator → Converter
Existing Tests	Has tests, no POM	Converter → Healer (if needed)
Failing Tests	Tests breaking	Healer
Expand Coverage	Some tests exist	Planner → Generator → Converter
Maintenance	Tests working	Healer (when needed)

Scenario 1: Starting from Scratch (New Project)

Use Case: You have a web application with no tests yet and want to build a production-ready test suite.

Step 1: Create Test Plan with Planner

Invoke: `@playwright-test-planner`

Prompt:

```
Create a comprehensive test plan for my web application at https://your-app.com

Please explore:
- User registration and login flows
- Main dashboard features
- Critical user workflows
- Edge cases and error scenarios

Save the test plan to specs/test-plan.md
```

What happens:

1. Agent opens your application in a browser
2. Explores the UI interactively
3. Identifies user flows and scenarios
4. Creates structured test plan document
5. Saves to `specs/test-plan.md`

Output Example:

```
# Test Plan: E-commerce Application

## Test Suite: User Authentication

### Test: Successful User Login
**Steps**:
1. Navigate to login page
2. Enter valid email
3. Enter valid password
4. Click login button
5. Verify dashboard is displayed

**Expected Results**:
- User is redirected to dashboard
- Welcome message displays user name
- Navigation menu is visible
```

Step 2: Generate Tests with Generator

Invoke: `@playwright-test-generator`

Prompt:

```
Generate Playwright tests from the test plan in specs/test-plan.md

Focus on:
- User Authentication suite
- Dashboard Features suite

Save tests in tests/ directory
```

What happens:

1. Agent reads your test plan
2. Executes each step in a real browser
3. Generates TypeScript test code

4. Creates `.spec.ts` files
5. Tests are ready to run

Output Example:

```
// tests/authentication.spec.ts
import { test, expect } from '@playwright/test';

test.describe('User Authentication', () => {
  test('successful user login', async ({ page }) => {
    await page.goto('https://your-app.com/login');
    await page.getByLabel('Email').fill('user@example.com');
    await page.getByLabel('Password').fill('password123');
    await page.getByRole('button', { name: 'Sign In' }).click();

    await expect(page).toHaveURL(/dashboard/);
    await expect(page.getByText('Welcome')).toBeVisible();
  });
});
```

Verify Tests Work:

```
npm test
```

Step 3: Convert to POM with Converter (Recommended)

Invoke: `@playwright-pom-converter`

Prompt:

Convert my Playwright tests to use Page Object Model pattern

Requirements:

- Remove all hardcoded credentials
- Create page objects for all pages
- Set up environment variables
- Use modern fixture pattern
- Generate documentation

Include:

- Login page
- Dashboard page
- Any other pages found in tests

What happens:

1. Agent scans all test files
2. Identifies hardcoded credentials (removes them)
3. Creates `.env.example` and security setup
4. Creates `BasePage.ts` with utilities
5. Generates page object classes
6. Creates test fixtures
7. Refactors tests to use page objects
8. Generates documentation (README, MIGRATION, etc.)
9. Validates all tests still pass

Output Structure:

```
|— .env.example          # Environment variables template
|— src/
|   |— pages/
|   |   |— BasePage.ts    # Base utilities
|   |   |— LoginPage.ts   # Login page object
|   |   |— DashboardPage.ts # Dashboard page object
|   |— fixtures/
|   |   |— test-fixtures.ts # Fixture definitions
|— tests/
|   |— authentication.spec.ts # Refactored tests
|— README.md             # Framework documentation
|— MIGRATION.md          # Migration guide
|— QUICK_START.md        # Quick reference
```

Output Example:

```
// src/pages/LoginPage.ts
import { BasePage } from './BasePage';

export class LoginPage extends BasePage {
  private readonly emailInput = this.page.getByLabel('Email');
  private readonly passwordInput = this.page.getByLabel('Password');
  private readonly loginButton = this.page.getByRole('button', { name:
'Sign In' });

  async navigate() {
    await this.navigateWithAuth(`${process.env.BASE_URL}/login`);
  }

  async login(email: string, password: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
}
```

```
// tests/authentication.spec.ts (refactored)
import { test, expect } from '../fixtures/test-fixtures';

test('successful user login', async ({ loginPage }) => {
  await loginPage.navigate();
  await loginPage.login(
    process.env.LOGIN_EMAIL!,
    process.env.LOGIN_PASSWORD!
  );
  await expect(loginPage.page).toHaveURL(/dashboard/);
});
```

Setup Environment:

```
# Copy template
cp .env.example .env

# Edit .env with your credentials
# LOGIN_EMAIL=your-email@example.com
# LOGIN_PASSWORD=your-password
# BASE_URL=https://your-app.com
```

Run Tests:

```
npm test
```

Step 4: Maintain with Healer (When Needed)

When to use: Tests start failing due to UI changes

Invoke: `@playwright-test-healer`

Prompt:

```
Fix failing tests in tests/authentication.spec.ts

The login button locator seems to be broken after recent UI changes.
```

What happens:

1. Agent runs all tests to identify failures
2. Debugs each failing test
3. Inspects browser console and network
4. Updates broken locators

5. Validates fixes work
6. Continues until all tests pass

Scenario 2: Converting Existing Tests to POM

Use Case: You already have Playwright tests but they're becoming hard to maintain.

Step 1: Assess Current State

Check what you have:

```
# List all test files
find tests -name "*.spec.ts"

# Check for hardcoded credentials
grep -r "password" tests/
grep -r "@.*\\.com" tests/
```

Step 2: Use Converter Directly

Invoke: @playwright-pom-converter

Prompt:

Convert all my existing Playwright tests to Page Object Model pattern

Current issues:

- Tests have hardcoded credentials
- Lots of duplicated selectors
- No page objects currently
- Tests in tests/ directory

Please:

1. Remove all hardcoded credentials and use environment variables
2. Create page objects for all pages
3. Set up modern fixture system
4. Refactor all tests
5. Generate documentation
6. Validate all tests pass after conversion

What happens:

1. Security audit (finds credentials)
2. Creates .env.example and .gitignore updates
3. Builds infrastructure (BasePage, fixtures)
4. Generates page objects by analyzing tests

5. Refactors tests to use page objects
6. Runs tests to validate
7. Creates documentation

Timeline:

- Small project (<20 tests): 15-30 minutes
- Medium project (20-50 tests): 30-60 minutes
- Large project (50+ tests): 1-2 hours

Step 3: Review and Test

Review generated files:

```
# Check page objects
ls -la src/pages/

# Check fixtures
cat src/fixtures/test-fixtures.ts

# Review environment setup
cat .env.example
```

Run tests:

```
npm test
```

If tests fail: Use Healer to fix issues

PROF

Step 4: Use Healer for Fixes (If Needed)

Invoke: `@playwright-test-healer`

Prompt:

```
Some tests are failing after POM conversion. Please fix them.
```

Scenario 3: Maintaining and Fixing Tests

Use Case: Tests are failing after application updates.

Direct Approach: Use Healer

Invoke: @playwright-test-healer

Prompt:

Fix all failing tests in my test suite

Recent changes:

- Login page UI was redesigned
- New button styles and IDs
- Form layout changed

Please:

1. Run all tests
2. Fix broken locators
3. Update page objects if they exist
4. Ensure all tests pass

What happens:

1. Runs entire test suite
2. Identifies failing tests
3. Debugs each failure (console, network, snapshots)
4. Updates locators in tests OR page objects
5. Re-runs to validate
6. Continues until all pass

Targeted Fix

Invoke: @playwright-test-healer

Prompt:

Fix the failing test: tests/authentication.spec.ts - "successful user login"

Error message: "Locator not found: button[name='Sign In']"

Scenario 4: Expanding Test Coverage

Use Case: Add new tests to existing POM-based suite.

Step 1: Create Test Plan for New Features

Invoke: @playwright-test-planner

Prompt:

Create test plan for new checkout flow at <https://your-app.com/checkout>

Please explore:

- Adding items to cart
- Checkout process
- Payment form
- Order confirmation

We already have tests for login and dashboard.
Save plan to `specs/checkout-flow.md`

Step 2: Generate Tests with POM Support

Invoke: `@playwright-test-generator`

Prompt:

Generate tests from `specs/checkout-flow.md`

Important:

- We are using Page Object Model pattern
- Import from `'../fixtures/test-fixtures'`
- Use existing `LoginPage` fixture for authentication
- Create new page objects if needed (`CartPage`, `CheckoutPage`)

Save tests to `tests/checkout.spec.ts`

Note: Generator should recognize POM pattern and generate compatible code.

If it doesn't generate POM-style tests, use Converter:

PROF

Step 3: Convert New Tests (If Needed)

Invoke: `@playwright-pom-converter`

Prompt:

Convert the new tests in `tests/checkout.spec.ts` to use POM pattern

We already have:

- `BasePage`, test fixtures infrastructure
- `LoginPage`, `DashboardPage` objects

Please:

- Create new page objects (`CartPage`, `CheckoutPage`, etc.)

- Add them to fixtures
- Refactor new tests to use them
- Keep consistent with existing POM structure

Step 4: Fix Any Issues

Invoke: `@playwright-test-healer`

Prompt:

Run and fix any issues in tests/checkout.spec.ts

Scenario 5: Complete CI/CD Workflow

Use Case: Setting up automated testing in CI/CD pipeline.

Initial Setup (One-time)

1. Create Test Suite

`@playwright-test-planner` → Create comprehensive test plan
`@playwright-test-generator` → Generate all tests
`@playwright-pom-converter` → Convert to POM with CI/CD setup

The Converter agent creates `.github/workflows/playwright.yml` automatically.

2. Add Secrets to GitHub

PROF

Go to: **Repository Settings** → **Secrets and Variables** → **Actions**

Add secrets:

- `LOGIN_EMAIL`
- `LOGIN_PASSWORD`
- `BASE_URL`
- `HTTP_AUTH_USERNAME` (if needed)
- `HTTP_AUTH_PASSWORD` (if needed)

3. Enable Workflow

Commit and push:

```
git add .
git commit -m "Add Playwright test framework with POM"
git push
```

Tests run automatically on every push!

Ongoing Maintenance

When CI Tests Fail

Option 1: Fix Locally First

```
# Pull latest changes
git pull

# Run tests locally
npm test

# If they fail, invoke Healer
```

Invoke: `@playwright-test-healer`

```
Fix failing tests locally. CI pipeline is failing with:
[paste error from GitHub Actions]
```

```
# After fixes
git add .
git commit -m "Fix failing tests"
git push
```

PROF

Option 2: Debug CI Artifacts

1. Download failed test artifacts from GitHub Actions
2. Review screenshots and traces
3. Invoke Healer with context:

Invoke: `@playwright-test-healer`

```
Fix tests based on CI failure

Error from CI:
```

- Test: "user can checkout"
- Error: "Timeout waiting for selector"
- Screenshot shows button has new class name

Please update the locators.

Adding New Tests to CI

Just follow normal workflow:

```
@playwright-test-planner → Plan new features
@playwright-test-generator → Generate tests
@playwright-pom-converter → Ensure POM consistency
git push → Tests run automatically
```

Best Practices

1. Always Start with Planning

Even for small features, use Planner first:

```
@playwright-test-planner Create test plan for [feature]
```

Benefits:

- Better test coverage
- Clear documentation
- Identifies edge cases
- Team alignment

PROF

2. Use POM for Projects with 10+ Tests

If you have more than 10 tests, use Converter:

```
@playwright-pom-converter Convert to POM pattern
```

Benefits:

- Easier maintenance
- Less duplication
- Better security

- Scalable architecture
-

3. Use Healer Instead of Manual Fixes

When tests break, don't fix manually:

```
@playwright-test-healer Fix failing tests
```

Benefits:

- Faster fixes
 - Better locators (uses best practices)
 - Updates documentation
 - Consistent patterns
-

4. Regularly Update Test Plans

When app changes significantly:

```
@playwright-test-planner Update test plan for [changed feature]
@playwright-test-generator Regenerate tests from updated plan
```

5. Version Control Test Plans

Commit test plan files:

```
git add specs/*.md
git commit -m "Update test plans"
```

Benefits:

- Track test coverage over time
 - Review changes in PRs
 - Documentation stays current
-

6. Environment Variables for Everything

Never hardcode:

- ✕ Credentials
- ✕ URLs

- ✗ API keys
- ✗ Test data emails

Always use `.env`:

- ✓ `process.env.LOGIN_EMAIL`
- ✓ `process.env.BASE_URL`
- ✓ `process.env.API_KEY`

7. Run Tests Before Pushing

```
# Always run locally first
npm test

# If failures:
# @playwright-test-healer Fix failing tests

# Then push
git push
```

Troubleshooting

Issue: Agent Not Found

Problem: `@playwright-test-planner` not recognized

Solution:

```
# Ensure you ran init-agents
npx playwright init-agents --loop=vscode

# Restart VS Code
# Reload window: Cmd/Ctrl + Shift + P → "Reload Window"
```

Issue: Environment Variables Not Working

Problem: Tests fail with "LOGIN_EMAIL not configured"

Solution:

```
# Check .env exists
ls -la .env

# If not, create from template
```

```
cp .env.example .env
```

```
# Edit .env and add your credentials  
nano .env
```

Issue: Page Objects Not Imported

Problem: TypeScript error "Cannot find module '../pages/LoginPage'"

Solution:

```
# Check file exists  
ls -la src/pages/LoginPage.ts  
  
# Check fixtures are updated  
cat src/fixtures/test-fixtures.ts  
  
# If missing, invoke Converter again  
# @playwright-pom-converter Fix page object imports
```

Issue: Tests Pass Locally But Fail in CI

Problem: CI fails but local tests pass

Common causes:

1. **Missing secrets:** Check GitHub Actions secrets
2. **Different environment:** CI uses headless, different timing
3. **Network issues:** CI might be slower

Solution:

```
# Test in CI-like environment  
npm test -- --project=chromium  
  
# If still issues:  
# @playwright-test-healer Fix tests failing in CI
```

Issue: Converter Creates Wrong Page Objects

Problem: Page objects don't match your application

Solution:

Invoke: @playwright-pom-converter

Re-generate page objects with corrections:

Issues:

- LoginPage should use email field, not username
- Dashboard has new widgets to include

Please:

1. Update LoginPage.ts to use email field
2. Add new dashboard widgets to DashboardPage.ts
3. Update tests accordingly

Issue: Too Many Page Objects

Problem: Converter created too many page objects

Solution: Manually consolidate (or ask Converter to simplify)

Invoke: @playwright-pom-converter

Simplify page object structure

Current: 15 page objects for similar forms

Goal: Create 1 FormPage with reusable methods

Please consolidate and update tests.

Issue: Healer Can't Fix Test

Problem: Healer tries but test still fails

Solution: Provide more context

Invoke: @playwright-test-healer

Fix test with additional context:

Test: tests/checkout.spec.ts - "complete purchase"

Failure: "Payment button not found"

Additional info:

- Payment button only appears after form validation
- Need to fill all required fields first
- Button has dynamic ID based on payment method

Please update test logic.

Advanced Workflows

Workflow 1: A/B Testing Support

Scenario: Application has A/B tests, different users see different UI

Approach:

```
@playwright-test-planner Create test plan for both variants

@playwright-test-generator Generate tests for Variant A
@playwright-test-generator Generate tests for Variant B

@playwright-pom-converter Convert both test suites
```

Use environment variable to switch:

```
// .env
UI_VARIANT=A

// In tests
if (process.env.UI_VARIANT === 'A') {
  await expect(page.locator('.variant-a-button')).toBeVisible();
} else {
  await expect(page.locator('.variant-b-button')).toBeVisible();
}
```

PROF

Workflow 2: Multi-Language Testing

Scenario: Test application in multiple languages

Approach:

```
@playwright-test-planner Create test plan (in English)

@playwright-test-generator Generate tests for English

@playwright-pom-converter Convert to POM
```

Add language support to page objects:

```
// src/pages/LoginPage.ts
export class LoginPage extends BasePage {
  private getLoginButton() {
    const lang = process.env.TEST_LANGUAGE || 'en';
    const labels = {
      en: 'Sign In',
      es: 'Iniciar Sesión',
      fr: 'Se connecter'
    };
    return this.page.getByRole('button', { name: labels[lang] });
  }
}
```

Workflow 3: Visual Regression Testing

Scenario: Add visual testing to existing suite

Approach:

1. Generate tests with Planner + Generator
2. Convert with POM Converter
3. Add visual assertions:

```
test('dashboard layout', async ({ dashboardPage }) => {
  await dashboardPage.navigate();

  // Functional assertions (from generated tests)
  await expect(dashboardPage.header).toBeVisible();

  // Add visual regression
  await expect(page).toHaveScreenshot('dashboard.png');
});
```

PROF

If visual tests fail, use Healer to investigate.

Summary: Choosing the Right Agent

Task	Agent	Command
Explore app & plan tests	Planner	@playwright-test-planner
Create test code	Generator	@playwright-test-generator
Make tests maintainable	Converter	@playwright-pom-converter
Fix broken tests	Healer	@playwright-test-healer

Task	Agent	Command
Add new feature tests	Planner → Generator	Both agents
Refactor existing tests	Converter	@playwright-pom-converter
Debug CI failures	Healer	@playwright-test-healer
Update after UI changes	Healer	@playwright-test-healer

Quick Reference: Common Commands

Starting New Project

```
@playwright-test-planner Create test plan for https://my-app.com
@playwright-test-generator Generate tests from specs/test-plan.md
@playwright-pom-converter Convert all tests to POM pattern
```

Converting Existing Tests

```
@playwright-pom-converter Convert my tests to use Page Object Model with
environment variables
```

Fixing Failures

```
@playwright-test-healer Fix all failing tests
```

Adding New Tests

```
@playwright-test-planner Create test plan for [new feature]
@playwright-test-generator Generate tests from latest plan
```

Updating After Changes

```
@playwright-test-healer Update tests after UI changes
```

Getting Help

Agent-Specific Help

Each agent responds to help requests:

```
@playwright-test-planner What can you do?  
@playwright-test-generator Explain your workflow  
@playwright-pom-converter What is Page Object Model?  
@playwright-test-healer How do you fix tests?
```

Documentation Files

After using agents, check generated documentation:

- [README.md](#) - Framework overview
- [MIGRATION.md](#) - Conversion guide
- [QUICK_START.md](#) - Quick reference
- [specs/*.md](#) - Test plans

Community & Support

- [Playwright Documentation](#)
- [Playwright Discord](#)
- [GitHub Issues](#)

Conclusion

The Playwright Agent Framework provides a complete solution for test automation:

1. **Planner** helps you think through test coverage
2. **Generator** turns plans into working code
3. **Converter** makes tests maintainable and secure
4. **Healer** keeps tests running as apps evolve

PROF

Recommended Flow for All Projects:

```
Planner → Generator → Converter → (Healer as needed)
```

This ensures you have:

- ✓ Comprehensive test coverage (Planner)
- ✓ Working test code (Generator)
- ✓ Maintainable architecture (Converter)
- ✓ Reliable tests over time (Healer)

Start your journey today:

```
@playwright-test-planner Let's create a test plan for my application!
```

Happy Testing! 🐛